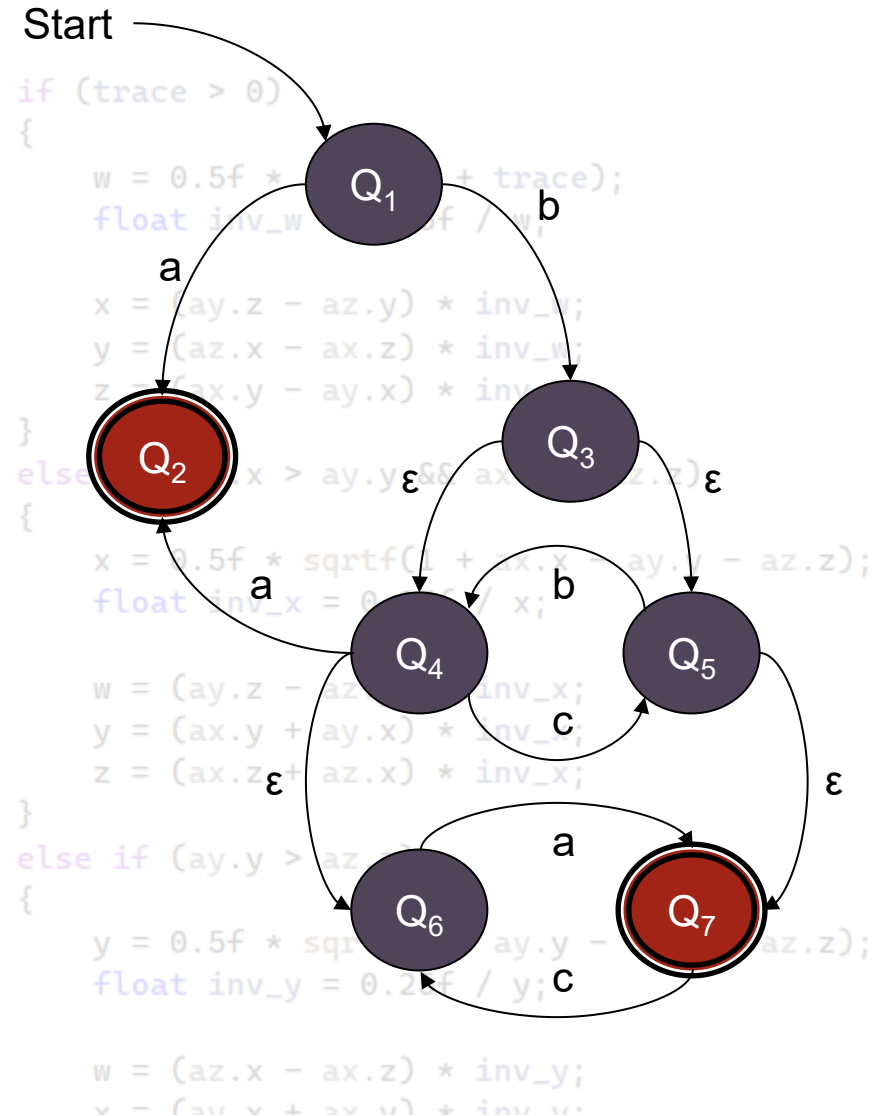


Algorithmen und Datenstrukturen

3 – Objektorientierte Programmierung

Tobias Werner M. Sc.
Uni Bayreuth - Angewandte Informatik 3
Leibniz-FH Hannover
Duale Hochschule Sachsen
tobias.werner@uni-bayreuth.de



Einleitung

Heranführung

- Was ist ein Objekt?
- Auf dem Schienennetz der deutschen Bahn sind zeitgleich 3 ICE-Züge unterwegs.
 - Was haben die Züge gemeinsam?
 - Worin unterscheiden sich die Züge?
- Warum ist es allgemein sinnvoll, die Haustüre abzusperren?
- Wie entscheiden Sie, wem Sie einen Schlüssel für die Wohnung geben?

Kompetenzen

- in C++ mit Klassen und Methoden arbeiten
- Objekte mit Konstruktoren bzw. Destruktoren anlegen und abwickeln
- robustere Programmierung mit Zugriffsspezifizierern und `friends`

Vortragsübersicht

3 – Objektorientierte Programmierung

- Objektorientierung als Konzept
- Klassen
- Konstruktoren und Destruktoren
- Zugriffsspezifizierer

Objektorientierung als Konzept (1)

Benutzerdefinierte Datentypen in C

```
struct enemy
{
    const char* name;
    int health;
    weapon* current_weapon;
    float position_x, position_y;
};

void main()
{
    struct enemy e1, e2;
    e1.name = "godfrey";
    e2.name = "malenia";
    printf("%s %s", e1.name, e2.name);
}
```

```
// zum Vergleich ohne struct:
void main()
{
    const char* e1_name;
    int e1_health;
    weapon* e1_current_weapon;
    float e1_position_x, e1_position_y;
    const char* e2_name;
    // ... viel Schreibarbeit ...
}

// gibt zwei verschiedene Namen aus
```

- definiere eigene Datentypen mit `struct`-Schlüsselwort
 - übersichtliche Bündelung von immer gemeinsam auftretenden Variablen
 - enthaltene Variablen sind *Member* (dt. *Felder*) der `struct`
 - *Instanziierung* der `struct` erzeugt eine *Instanz* (d. h. Variable) mit diesem Datentyp
 - Instanz enthält unabhängige Teilvariablen für alle `struct`-Inhalte

Objektorientierung als Konzept (2)

Operationen auf benutzerdefinierten Datentypen

```
typedef struct enemy_  
{  
    const char* name;  
    int health;  
    weapon* current_weapon;  
    float position_x, position_y;  
} enemy;                                     // erlaubt Verzicht auf struct-Schlüsselwort  
                                             // bei der Instanziierung  
  
void drop_weapon(enemy *e)  
{  
    if (!e->current_weapon) return;  
    e->current_weapon->carried = false;  
    e->current_weapon->position_x = e->position_x;  
    e->current_weapon->position_y = e->position_y;  
    e->current_weapon = nullptr;  
}
```

Praxisbeobachtung:

häufig haben benutzerdefinierte Datentypen zugeordnete *Operationen*

(d. h. Funktionen, die hauptsächlich auf einer Instanz des Datentypen arbeiten)

- Umsetzung in C mittels Funktionen und Zeiger als Parameter
- geht das schöner?

Objektorientierung als Konzept (3)

Objektorientierung als Lösungsansatz

```
class enemy
{
    weapon* current_weapon;
    float position_x, position_y;
    void drop_weapon()
    {
        if (!this->current_weapon) return;
        this->current_weapon->carried = false;           // this-Zeiger verweist
        this->current_weapon->position_x = this->position_x; // auf genutzte Instanz
        this->current_weapon->position_y = this->position_y; // für den Aufruf
        this->current_weapon = nullptr;
    }
};
// ...
enemy e;           // instanziiere Objekt zu Klasse
e.drop_weapon();   // führe Methode auf Objekt aus, this zeigt automatisch auf e
```

- *Klasse* ist ein Datentyp, der Member und zugehörige *Methoden* (Operationen) bündelt
- *Objekt* einer Klasse ist eine Instanz einer Klasse (d. h. eine Variable vom Klassendatentyp)
 - typische Realisierung (z. B. C++): Methoden erhalten `this`-Zeiger auf genutzte Instanz
- *Objektorientierung* (bzw. *objektorientierte Programmierung*) ist Programmierparadigma unter Verwendung von Klassen und Objekten

Objektorientierung als Konzept (4)

Syntaxzucker als wesentlicher Teil der Objektorientierung

```
class enemy
{
    weapon* current_weapon;
    float position_x, position_y;
    void drop_weapon()
    {
        if (!current_weapon) return;
        current_weapon->carried = false;           // Syntaxzucker:
        current_weapon->position_x = position_x;     // automatischer Zugriff
        current_weapon->position_y = position_y;     // über this-Zeiger

        weapon* current_weapon;
        current_weapon = nullptr;                   // Zugriff jetzt auf lokale Variable,
                                                    // statt auf this->current_weapon
    }
};
```

- *Syntaxzucker* ist Sprachfeature, das übersichtlichere Programme ermöglicht, indem für bestehende Features eine kürzere Form angeboten wird
→ Beispiel C++: Zugriff auf Variable verwendet `this`-Zeiger womöglich automatisch

Faustregel: Für bessere Lesbarkeit in C++ kein unnötiges `this->` verwenden!

Vortragsübersicht

3 – Objektorientierte Programmierung

- Objektorientierung als Konzept
- Klassen
- Konstruktoren und Destruktoren
- Zugriffsspezifizierer

Klassen (1)

Klassendeklaration

```
class anonymous;           // compiler learns anonymous is some class

// ... no definition here ...

anonymous a;              // compiler error: class constructors unknown
a.name = "Kevin";         // compiler error: class members unknown
a.speak();                // compiler error: class members unknown
int size = sizeof(anonymous); // compiler error: class size unknown

// ...

void unmask(anonymous *a) // OK: pointer doesn't need class internals
{
    if (!a) throw invalid_person();
    std::cout << a->name << std::endl; // compiler error: class members unknown
    anonymous* next_a = a + 1;         // compiler error: pointer arithmetics need size
}
```

- *Klassendeklaration* legt Klassennamen fest
- Klassendetails (z. B. Größe und Member) unbekannt → Compile-Fehler bei Verwendung
- Klassenverwendung ohne Details erlaubt → Zeiger auf Instanzen weitgehend ok
→ *Vorwärtsdeklarationen* (d. h. Klassendeklaration ohne Rumpf, vgl. nächste Folie)
statt `#include`, wo möglich, um Header-Einlesen zu sparen

Klassen (2)

Klassendefinition

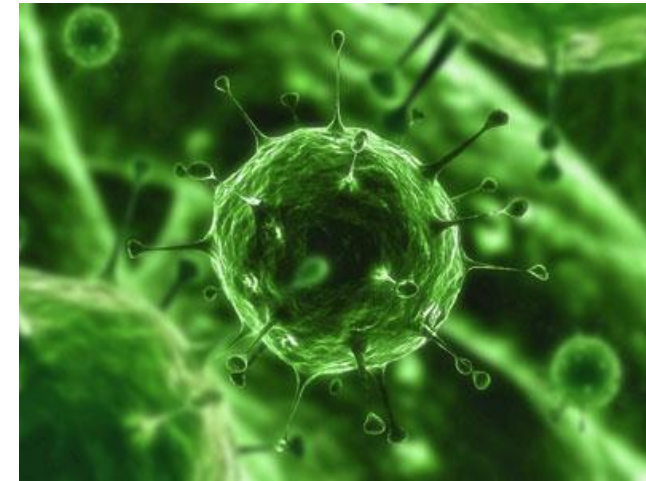
```
class virus
{
    virus(const string& name, const dna_sequence& s);
    ~virus();

    virus breed(const virus& other);

    std::string name;
    dna_sequence s;
};
// ...

virus va, vb;
std::cout << va.name << std::endl;
std::cout << sizeof(va) << std::endl;
va.breed(vb);
```

// OK, constructors known
// OK, members known
// OK, size known
// OK, methods of va known



[<http://static.ddmcdn.com/gif/light-virus-1.jpg>]

- *Klassendefinition* ist Klassendeklaration mit zugehörigem Klassenrumpf und legt Membermethoden sowie Membervariablen fest
 - vollständiger Zugriff auf Klasse und Instanzen nach Definition möglich
 - nur eine Klassendefinition pro Klasse und pro Übersetzungseinheit erlaubt (Ausnahme: Templates → evtl. späterer Vortrag!)

Klassen (3)

Mehrfache Klassendefinitionen

experiment.hpp

```
// ...  
class virus; // reicht für infect()-Deklaration  
void infect(const virus& sample);  
// ...
```

experiment.cpp

```
// Hier könnte man auf #include "experiment.hpp" verzichten!  
class virus { std::string name; dns_sequence s; };  
class virus { std::string name; dns_sequence s; }; // Compiler-Fehler, obwohl gleich  
void infect(const virus& sample)  
    { std::cout << "Started infection: " << sample.name; }
```

simulation.cpp

```
#include "experiment.hpp"  
class virus { dns_sequence s; }; // kein Compiler- oder Linker-Fehler  
int main() { infect(virus()); } // undef. Vh., vermutlich segfault in infect()
```

- doppelte Definition in einer .cpp → Compiler-Fehler
- doppelte, unterschiedliche Definitionen über .cpp-Grenzen → undef. Vh.

Klassen (4)

Deklaration von Membermethoden

```
class vector                                     // gleichzeitig: Deklaration der Klasse vector
{
    float length();

    vector dot_product(vector&);                // Klasse vector ist hier bekannt
    vector dot_product(vector& s2);             // compiler error: invalid overload
    vector dot_product(vector& other);          // compiler error: invalid overload

    // ...
};
```

- *Methodendeklaration* (ohne Rumpf auch *Vorwärtsdeklaration*) für Membermethoden
legt Methodennamen, Rückgabebetyp und Parameter fest
 - Parameternamen weithin beliebig, dürfen auch entfallen
 - darf nur einmal pro Klassenrumpf auftreten
 - genügt, um Methode aufzurufen
 - üblicherweise in Header-Datei (.hpp)
 - benötigt als Vorwärtsdeklaration eine Definition außerhalb der Klasse

Klassen (5)

Definition von Membermethoden außerhalb der Klasse

geo3d.hpp

```
// ...  
class vector  
{  
    vector dot_product(const vector& other);  
    /* ... */  
};
```

geo3d.cpp

```
#include "geo3d.hpp"           // Header ist hier fast zwingend erforderlich,  
                               // sonst müsste die ganze Klasse wiederholt werden  
vector vector::dot_product(const vector& v2) // Klasse vector mit Inhalt ist bekannt  
{ return vector(x * v2.x, y * v2.y, z * v2.z); }
```

- Definition außerhalb der Klasse über `klassenname::methodensname`
 - dies ist in C++ das Standardvorgehen für Membermethoden
- mehrfache (selbst identische) Definitionen im Gesamtprogramm unzulässig
 - Verlagerung der Definition in exakt eine .cpp-Datei
 - **undef. Vh.** bei mehrfacher Definition (meist: Compiler- bzw. Linker-Fehler)
 - Ausnahme: Templates

Klassen (6)

Inline-Definition von Membermethoden

```
class vector
{
    float length()
    { return sqrt(x * x + y * y + z * z); }
    /* ... */
};
```

- Angabe in Klassenrumpf mitsamt Methodenrumpf
- mehrfache, identische Inline-Definitionen im Gesamtprogramm zulässig
(also in jeder .cpp, die den Header zu einer Klasse verwendet)
→ undef. Vh. wenn sich die Inline-Definitionen unterscheiden!

interne Realisierung nur für inline-definierte Methoden:

Code wird bei jedem Methodenaufruf ohne Sprungbefehl einkopiert (*inlining*)

→ spart auf alten Compilern und für kleine Methoden Rechenzeit ein

→ widersprüchlich: bei häufiger Nutzung evtl. größeres und langsames Programm
(*Caching* bzw. *Branch Prediction* der CPU werden damit umgangen)

→ moderne Compiler-Optimierungen machen Inlining ohnehin nach Lust und Laune

3 – Objektorientierte Programmierung

- Objektorientierung als Konzept
- Klassen
- Konstruktoren und Destruktoren
- Zugriffsspezifizierer

Konstrukturen und Destruktoren (1)

Konstrukturen als Konzept

```
class file
{
    file(const char* path, bool enable_write)    // Konstruktor mit Parametern
    {
        if (enable_write) file_handle = fopen(path, "wb+");
        else file_handle = fopen(path, "rb");
    }

    void read(char* buffer, int num_bytes)
    { ::read(file_handle, buffer, num_bytes); } // :: nutzt freie read()-Funktion
                                                // statt Methode file::read()

    FILE* file_handle;
};
// ...

file f("test.txt", false);    // Konstruktor-Aufruf
f.read(my_string, 16);        // file_handle-Member ist jetzt initialisiert
```

Beobachtung:

mehrere Instanzen einer Klasse brauchen meist ähnliche Logik,
um ihre Membervariablen zu initialisieren

→ *Konstruktormethode* (kurz *Konstruktor*) ist beim Erzeugen eines Objekts
aufgerufene Methode, welche die Initialisierung vornimmt

Konstruktor und Destruktoren (2)

Exkurs *Funktionsüberladung*

Idee:

- erlaube Funktionen mit gleichem Namen, aber unterschiedlichen Parametern
- bestimme aufgerufene Funktion anhand Zahl und Typ der Eingabeparameter
 - klappt nur bei eindeutiger Auflösung
 - Rückgabetyt wird bei Auflösung nicht berücksichtigt

Beispiele:

```
float interpolate(float t, float p1, float p2);           // (1) linear, float
float interpolate(float t, float p1, float p2, float p3); // (2) quadratisch, float
double interpolate(double t, double p1, double p2);      // (3) linear, double
float interpolate(double t, double p1, double p2);        // Comp.-Fehler, Überladung
// ...                                                    // nur mit Rückgabetyt

interpolate(0.3f, 0.2f, 0.8f);                           // Aufruf (1)
interpolate(0.3f, 0.2f, 0.8f, 1.2d);                      // Aufruf (2), Typumwandlung
interpolate(0.6d, 0.7d, 0.9d);                           // Aufruf (3)
interpolate(0.5d, 0.3f, 0.5d);                           // Comp.-Fehler, mehrdeutig
interpolate(0, 1, 2);                                     // Comp.-Fehler, mehrdeutig
```

Konstruktoren und Destruktoren (3)

Anwendung bei Konstruktoren

```
class image
{
    // Create image with sx, sy pixels and one color.
    image(int sx, int sy, const color& col);

    // Load image from given image file.
    image(const std::string& filename);

    // Put text on image.
    // Compiler-Fehler: doppelte Definition,
    // Überladung über Parameternamen nicht zulässig
    image(const std::string& text);

    // Put text on image or load from file.
    // aufgeblähter Code:
    // völlig unterschiedliche Funktionalitäten in einer Funktion
    enum operation_mode { from_file, from_text };
    image(const std::string& s, operation_mode m);
}
```



[<http://www.heise.de/autos/artikel/Die-skurrilsten-Eigenreparaturen-1717729.html?bild=5>]

- Konstruktoren sind spezielle Funktionen, lassen sich also je nach Bedarf überladen
- **Achtung: inflationäre Verwendung von Funktionsüberladung ist keine gute Idee**
 - Konstruktorparameter sollten recht direkt mit dem späteren Objekt zusammenhängen
 - alternativ freie Methoden zum erzeugen nehmen (vgl. `create_image` als Methode)

Konstruktor und Destruktoren (4)

Naive Konstruktor in C++

```
class storage
{
    storage(const std::string& url, float timeout)
        // hier automatisch Default-Konstruktion url_(), c_(), nach C++11 timeout_ = 0
    {
        url_ = url; // ersetze "" in variable url_ durch url
        c_ = database_connection(url_); // ersetze nicht verbundene Verbindung c_
        timeout_ = timeout; // Compiler-Fehler, Zuweisung zu const-Variable
    }

    std::string url_;
    database_connection c_;
    const float timeout_;
};
```

- in anderen Sprachen typische Konstruktion per Zuweisung hat in C++ andere Semantik:
 - Zuweisung zu bereits default-konstruierten Variablen
- je nach Situation...
 - ... langsam (z. B. Konstruieren und Ersetzen unnötiger Default-Werte)
 - ... nicht möglich (z. B. konstante Werte oder kein Default-Konstruktor vorhanden)

Konstruktor und Destruktoren (5)

Konstruktor-Initialisierungssyntax

```
// Initialisierungssyntax: Direkte Konstruktoraufrufe der Member,  
// für Definition des Konstruktors außerhalb der Klasse analog  
storage(const std::string& url, float timeout)  
:  
    url_(url),                // vermeide Standardwert ""  
    c_(url_),                 // vermeide nicht verbundene Verbindung c_  
    timeout_(timeout)         // const-Variablen müssen gleich richtig konstruiert werden  
{ }
```

- *(Konstruktor-)Initialisierungsliste* zwischen Konstruktorparametern und -rumpf
- Syntax: `: member_1(Parameter_1, Parameter_2), member_2(Parameter_3), /* ... */`
- direkter Aufruf von Member-Konstruktor mit jeweils passenden Parametern
 - kein evtl. teures Initialisieren unnötiger Standardwerte
 - erlaubt Verwendung von konstanten Klassenmembern
 - erlaubt Verwendung von Klassen ohne Default-Konstruktor
- Konstruktorinitialisierungsliste nicht mit `std::initializer_list` verwechseln!

Faustregel:

Bevorzuge immer die Initialisierungssyntax über Zuweisungen.

Konstrukturen und Destruktoren (6)

Unvollständige Initialisierungslisten

```
class gui
{
    std::string name;
    std::unique_ptr<font> f;
    std::unique_ptr<texture> background;
    int num_windows;
    gui() :
        // hier automatisch: name()
        f(new font()),
        background(load_texture())
        // hier automatisch ab C++11: num_windows(0)
    {
        name = "marker_scanning_results"; // Voraussetzung: name ist schon konstruiert
    }
};
```



[<https://www.ea.com>]

- Abarbeitung Initialisierungsliste in Memberreihenfolge
 - nicht aufgeführte Member werden default-konstruiert (sofern möglich)
oder (nur eingebaute Typen, nur ab C++11) auf 0 bzw. nullptr gesetzt
 - im Konstruktor-Rumpf sind alle Member bereits konstruiert
 - definiertes Verhalten bei Zugriff (z. B. Methodenaufruf) im Konstruktorrumpf
- Ausblick: Zuweisung ist Aufruf der Methode `operator =`

Konstruktor und Destruktoren (7)

Unsortierte Initialisierungslisten

```
class window
{
    scrollbar right, bottom;
    scollable_area a;

    window(const rect& size) :
        a(size, *this),           // Reihenfolge: right(a), bottom(a), a(size, *this),
        right(a),                // mit etwas Glück: Compiler-Warnung,
        bottom(a)                // aktuell eher: Zugriff auf uninit. Speicher von a
                                // durch Konstruktor von right → undef. vh.
    { /* ... */ }
};
```

- Deklarationsreihenfolge der Member legt Konstruktionsreihenfolge fest
→ Reihenfolge in Initialisierungsliste ist dagegen unerheblich!
- Problem: Zugriff auf nichtkonstruiertes Objekt möglich
→ meist kein Compiler-Fehler
(Grund: wechselseitige Verzeigerung von Membern)

Faustregel:

Nutze in Klassendefinition und Initialisierungsliste immer die gleiche Member-Abfolge.

Konstruktor und Destruktoren (8)

Konstruktor- vs. Zuweisungssyntax bei lokalen Variablen

```
struct rgb_color
{
    rgb_color();
    rgb_color(char r, char g, char b);
    rgb_color(const rgb_color& other);
};

rgb_color c = rgb_color(17, 23, 199);
```

- compilerabhängige Interpretation:

```
c(), rgb_color temp(17, 23, 199), c = temp
c(17, 23, 199)
```

(wortwörtlich aufgefasst, alter gcc)

(nach C++-Standard, Visual Studio)

- korrigierte Beispiele:

```
rgb_color c(17, 23, 199);
rgb_color c;
```

(ausdrücklich Konstruktor aufgerufen)

(Default-Konstruktor ohne Klammern)

Faustregel:

Nutze für Konstruktoraufrufe immer die Konstruktorsyntax.

(→ Ausnahme: spezieller Template-Code, außerhalb vom VL-Umfang)

Konstruktor und Destruktoren (9)

Default-Konstruktor

```
class gui
{
    std::string name;
    std::unique_ptr<font> f;
    std::unique_ptr<texture> background;
    int num_windows;

    // hier automatisch:
    // public: gui() : name(), f(), background(), num_windows(0) { }
};
```

- *Default-Konstruktor* ist Konstruktor ohne Parameter
 - erzeugt "Standardobjekt" (z. B. leeren Datencontainer)
 - sollte tendentiell vermieden werden (vgl. RAII aus späterer VL-Einheit)
- *automatisch erzeugter Default-Konstruktor*
 - wird automatisch unter einer Sammlung von Bedingungen erzeugt, z. B.:
 - wenn kein eigener Default-Konstruktor definiert wird,
 - wenn sonst kein anderer, eigener Konstruktor angegeben ist, ...
 - ruft die Default-Konstruktor aller Member in Deklarationsreihenfolge auf

Konstruktoren und Destruktoren (10)

Copy-Konstruktoren

```
class image
{
    image(const image& other) :           // nach Parameter: Copy-Konstruktor
        data_(other.data_),             // Der würde genau so automatisch erzeugt.
        size_x_(other.size_x_),         // Hinweis: Falls hier Member fehlen,
        size_y_(other.size_y_) { }      // läuft für die der Default-Konstruktor!

    std::vector<color> data_;
    int size_x_, size_y_;
};
```

Semantik:

- *Copy-Konstruktor* erzeugt **unabhängig löschbare** Kopie von Quellobjekt
→ in der Regel Umsetzung als (weitgehend) tiefe Objektkopie

Rahmenbedingungen:

- Übergabe des Parameters immer von gleichem Typ und per konstanter Referenz
→ wird von der STL (`std::vector` u. a.) intern so erwartet
→ Objekte in Containern können beliebig oft kopiert werden
- automatische Erzeugung weitgehend analog zum Default-Konstruktor

Konstruktor und Destruktoren (11)

Destruktoren

```
class car
{
    std::unique_ptr<engine> e;
    std::shared_ptr<blueprint> b;
    std::vector<wheel> ws;
    int *id;
    car() : /* ... andere Member füllen ... */ id(new int(15)) { }
    ~car() noexcept { }           // automatisch: ws.~std::vector(), b.~shared_ptr(),
};                               // e.~unique_ptr(), aber Leck über einen int
```

- *Destruktor* ist spezielle Membermethode `~Klassenname()`
- exakt ein Destruktor pro Klasse
- Aufruf bei Objektlöschung (Stack: automatisch, Heap: durch `delete` oder `delete[]`)
 - Ziel: abhängige, manuell verwaltete Ressourcen aufräumen oder freigeben
- am Ende automatischer Aufruf aller Member-Destruktoren in umgekehrter Reihenfolge
 - kein Handlungsbedarf bei automatisch verwalteten Ressourcen
- automatische Erzeugung mit leerem Rumpf, falls nicht selbst angegeben
 - Speicherlecks oder anderes Fehlverhalten für manuell verwaltete Ressourcen

Vortragsübersicht

3 – Objektorientierte Programmierung

- Objektorientierung als Konzept
- Klassen
- Konstruktoren und Destruktoren
- Zugriffsspezifizierer und Friends

Zugriffsspezifizierer und Friends (1)

Motivation Zugriffsspezifizierer

Beobachtung:

- in der Praxis müssen Instanzen einer Klasse *Invarianten* (Bedingungen) erfüllen, zum Beispiel:
 - Zeiger-Membervariable `p` ist nicht `nullptr`
 - wenn `in_air = true` ist, muss `air_speed > 0` sein
 - zu jeder Klasseninstanz muss eine geöffnete Netzwerkverbindung existieren
- Code (z. B. Membermethoden, Anwendercode außerhalb der Klasse) kann sich auf die Invarianten verlassen (z. B. für bessere Lesbarkeit oder Robustheit)
- über längere Softwareentwicklung wird alter Code im Detail vergessen
 - Prüfung und Erhaltung von Invarianten außerhalb der Klasse geht leicht schief (z. B. mehrere Entwickler, triviale Tippfehler, Fehler durch redundanten Code)
 - in C++: häufig (nicht immer) **undef. Vh.** als Folge!

→ **Wir brauchen eine Lösung, die Invarianten auf höherer Ebene erzwingbar macht!**

Zugriffsspezifizierer und Friends (2)

Klasseninvarianten ohne Zugriffsspezifizierer

```
struct image
{
    int size_x, size_y;
    std::vector<color> data;

    image(int size_x, int size_y, const std::vector<color>& data)
        : size_x(size_x), size_y(size_y), data(data)
    {
        if (data.size() != size_x * size_y)           // erzwinge Klasseninvariante:
            throw invalid_parameter();                 // Breite * Höhe = Zahl der Pixel
    }

    color& at(int x, int y)
    {
        if (x < 0 || x >= size_x || y < 0 || y >= size_y)
            throw bounds_exceeded();                  // Zugriffsprüfung,
                                                    // falls Klasseninvariante besteht:
                                                    // niemals undef. Vh.

        return data[y * size_x + x];
    }
};

std::vector<color> colors(64);
image i(8, 8, colors);                               // Klasseninvariante erzwungen
if (i.size_y == 100)                                 // Klasseninvariante ausgehebelt
    i.at(0, 90) = color(1, 0, 0);                   // undef. Vh.
```

Zugriffsspezifizierer und Friends (3)

Klasseninvarianten mit Zugriffsspezifizierern

```
class image
{
    private:
        int size_x, size_y;
        std::vector<color> data;

    public:                                // Alle public-Methoden erhalten die Klasseninvarianten!
        image(int size_x, int size_y, const std::vector<color>& data)
            : size_x(size_x), size_y(size_y), data(data)
        {
            if (data.size() != size_x * size_y)
                throw invalid_parameter();
        }
        color& at(int x, int y)
        {
            if (x < 0 || x >= size_x || y < 0 || y >= size_y)
                throw bounds_exceeded();
            return data[y * size_x + x];
        }
};

std::vector<color> colors(64);
image i(8, 8, colors);
if (i.size_y == 100)                                // Klasseninvariante erzwungen
    i.at(0, 90) = color(1, 0, 0);                  // Compiler-Fehler, private, kein Zugriff
```

Zugriffsspezifizierer und Friends (4)

Zugriffsspezifizierer in C++

- Zugriffsspezifizierer definieren Zugriffsrechte
- Zugriffsspezifizierer gelten blockweise
- Standardzugriff `class` → `private`, `struct` → `public`

`public`: Zugriff von überall möglich

→ Methoden und Invarianten dokumentieren!

→ Zugriff über `public`-Schnittstelle erzwingt so gut es geht den Erhalt der Invarianten
(z. B. wenden Setter-Methoden nur gültige Werte auf Instanzen an)

`protected`: Zugriff nur von innerhalb der Klasse und ableitenden Klassen

→ Vererbung ist nicht Teil dieser Vorlesung

`private`: Zugriff nur von innerhalb der Klasse möglich

→ versteckte Hilfsmethoden und Membervariablen

Faustregel:

Nutze Zugriffsspezifizierer, um private Details einer Klasse zu verstecken.

Zugriffsspezifizierer und Friends (5)

Zugriffsspezifizierer und Setter- bzw. Getter-Methoden

```
class window
{
    private:
        int size_x_, size_y_;

    public:
        int& size_x() { return size_x_; }           // stupid
        void set_size_x(int size_x) { size_x_ = size_x; } // stupid
        void set_size_y(int size_y)                // OK
        {
            if (size_y < 0 || size_y > desktop_size) throw bounds_exceeded();
            size_y_ = size_y;
        }
        /* ... */
};
```

Faustregel:

Zugriffsspezifizierer nicht durch Etteritis aushebeln!

→ Ausnahme: Setter-Methoden mit zusätzlicher Logik

Zugriffsspezifizierer und Friends (6)

Zugriffsspezifizierer ohne Invarianten

```
class vector
{
    public:
        void set_x(float x) { x_ = x; }
        float x() { return x_; }
        void set_y(float y) { y_ = y; }
        float y() { return y_; }
        void set_z(float z) { z_ = z; }
        float z() { return z_; }

    private:
        float x_, y_, z_;
};
```

- Schnittstellentrennung ohne Invarianten ist Unfug, daher besser:

```
class vector { public: float x, y, z; };    // Jedes x, y, z ist gültiger Vektor!
```

Faustregel:

Zugriffsspezifizierer nur zum Erhalt von Invarianten einsetzen!



[<http://www.mspaintadventures.com>]

Zugriffsspezifizierer und Friends (7)

Alternative zur Invarianten-Erhaltung mit Zugriffsspezifizierern

```
struct image
{
    const int size_x, size_y;
    const std::vector<color> data;

    image(int size_x, int size_y, const std::vector<color>& data)
        : size_x(size_x), size_y(size_y), data(data)
        { if (data.size() != size_x * size_y) throw invalid_parameter(); }

    const color& at(int x, int y) { /* ... wie gehabt ... */ }
    /* ... */
};

image i(12, 12, std::vector<color>(144));
i.data.resize(153); // compiler error: data is const
```

- nicht-änderbare Objekte (z. B. alle Member `const`, vgl. Java `lang.string` und `final`)
 - validiere Invarianten nur im Konstruktor
 - Invarianten bleiben durch nicht-änderbare Objekte durchgehend erfüllt
 - Zugriffsspezifizierer in der Regel unnötig
 - Objekt-Änderungen gehen nur über teure Objektkopie!

Zugriffsspezifizierer und Friends (8)

Breite Public-Schnittstelle bei zusammenarbeitenden Klassen

```
struct node
{ node *left, *right; element e; }; // Hier: left, right, e sind public, um Zugriff
                                     // durch set und greatest_element() zuzulass
class set
{
    node* find_node(node& pivot, const element& e)
    {
        if (e < pivot.e)
            return pivot.left ? find_node(*pivot.left, e) : nullptr;
        if (e > pivot.e)
            return pivot.right ? find_node(*pivot.right, e) : nullptr;
        return &pivot;
    }
    /* ... */
};

const element& greatest_element(const node& n)
{ return n.right ? greatest_element(*n.right) : n.e; }
```

- zusammenarbeitende Klassen (z. B. in einem Modul) haben viele Abhängigkeiten
- Arbeit nur mit `public` und `private` erfordert breite `public`-Schnittstelle
- Anwender kann eigentlich `private` Klassendetails verändern
(im Beispiel: `left`- und `right`-Zeiger sowie `e` in den Knoten der Baumstruktur)

Zugriffsspezifizierer und Friends (9)

Friending zum Aufbrechen der Private-Schnittstelle als Alternative

```
class set
{ /* ... wie gehabt, mit Anpassung an andere Membernamen ... */ };

const element& greatest_element(const node& n)
{ return n.right_ ? greatest_element(*n.right_) : n.e_; }    // Zugriff auf privates

class node
{
    friend class set;
    friend const element& greatest_element(const node& n);

public:
    const element& e() { return e_; }
private:
    node *left_, *right_;
    element e_;
};
```

- Freunde dürfen auf private Elemente zugreifen
- öffentliche Schnittstelle auf anwenderrelevante Methoden beschränkt
- Anwenderzugriff auf private Klassendetails nicht möglich
(im Beispiel: Zugriff auf `left_`- bzw. `right_`-Zeiger von Baumknoten verboten,
Zugriff auf Wert `e_` nur über lesenden Zugriffsweg mittels Getter `e()`)

Zugriffsspezifizierer und Friends (10)

Friending in C++

- Syntax:
 `friend class classname;`
 `friend methodendeklaration;`
- in Klassenrumpf außerhalb anderer Inhalte, sonst beliebig
- Friend-Klasse oder Friend-Methode sollte bereits deklariert sein
- Freundschaften sind nicht transitiv oder symmetrisch
- Modulkonzept beachten:
 - Friends nur innerhalb eines geschlossenen Moduls vergeben
 - wichtige Klassen (`main_window`) nicht an unwichtige (`gui_button`) freunden
 - so wenige Friends wie möglich – nicht übertreiben!
 - Beispiel: `node ← iterator`, `node ← map`, `iterator ← map`

Faustregel:

Verwende friends, um öffentliche Schnittstellen auf Anwender zu beschränken.

Zusammenfassung

Objektorientierte Programmierung

- objektorientierte Programmierung verwendet Klassen und Objekte
- Methoden arbeiten auf Objekten einer Klasse
- Umsetzung in C++ mit `class`-Schlüsselwort analog zu `structs` aus C
- Deklaration, Vorwärtsdeklaration, Definition von Klassen
- Deklaration (in `.hpp`) und Definition (in `.cpp`) von Methoden
- auf One-Definition-Rule achten
- Konstruktoren initialisieren Objekte
- Konstruktoren lassen sich bei Bedarf überladen
- Destruktoren löschen Objekte
- Zugriffsspezifizierer schränken Zugriff auf Membervariablen und -methoden ein
- Zugriffsspezifizierer lassen sich durch `friend` aufbrechen